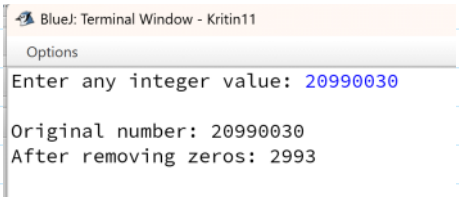
Constructors

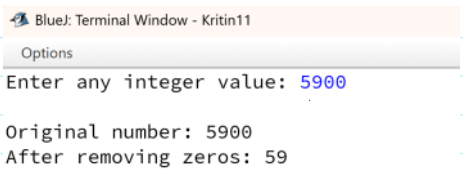
05 May 2026 05:27

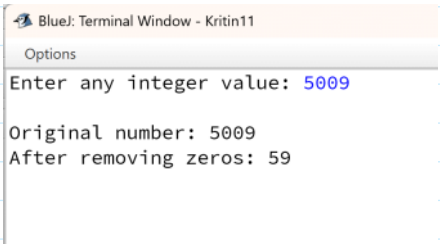
```
package Loops;
import java.util.*;

/**
 * Write a description of class Q9UsingInt here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Q9UsingInt
{
    //instance variables
    private int num, newNum;
    //parametrized constructor
    public Q9UsingInt(int num){
        this.num = num;
    }

    public int reverse(int n){
        int rev = 0;
        for(;n > 0; rev = rev * 10 + n % 10, n /= 10);
        return rev;
    }

    public int removeZero(int n){
        int digit, newNum = 0;
        for(;n > 0; n /= 10){
            digit = n % 10;
            if(digit > 0){
                newNum = newNum * 10 + digit;
            }
        }
        return reverse(newNum);
    }

    public void display(){
        newNum = removeZero(num);
        System.out.print("\nOriginal number: " + num +
            "\nAfter removing zeros: " + newNum);
    }

    public static void main(String[] args){
        System.out.print("\fEnter any integer value: ");
        int n = new Scanner(System.in).nextInt();
        Q9UsingInt ob = new Q9UsingInt(n);
        ob.display();
    }
}
```

In Java, a constructor is a special method used to initialize objects when a class is created.

Output:







What is a Constructor in Java?

A constructor in Java is a block of code like the method. It is called when an instance of a class is created. At the time of calling constructor, memory for the object is allocated in the memory. It is a special type of method which is used to initialize the object.

Every time when we create an object by using the **new** keyword, it calls a default constructor. If there is no constructor available in the class. In such case Java compiler provides a default constructor by default.

Note: It is called constructor because it constructs the values at the time of object creation. It is not necessary to write a constructor for a class. Because Java compiler creates a default constructor if your class does not have any.

Rules for creating Java Constructor

1. **Constructor name must be same as the class name:** The constructor must have the same name as the class. This helps Java compiler identify the constructor and use it while creating an object.

   Syntax:

```
class class_name{
   class_name(){
      …
   }
}
```

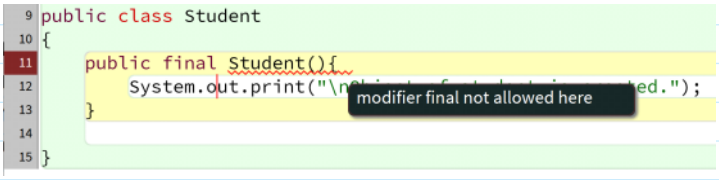
   For example:

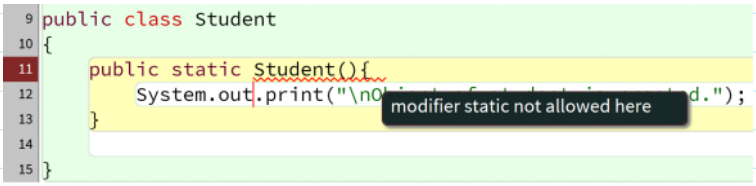
```
public class Student
{
    public Student(){ //Constructor
        …
    }
     …
}
```

2. **A constructor must have no explicit return type:** A constructor does not return any value, not even void. If you specify a return type, it will be treated as a normal method, not a constructor. For example:

```
public class Student
{
    public void Student(){ //Method not a Constructor, return type is mentioned
        System.out.print("\nObject of student is created.");
    }

}
```

3. **A Java constructor cannot be abstract, static, final, or synchronized:** Constructors are used to create an initialize object, so Java does not allow these modifiers with constructors.

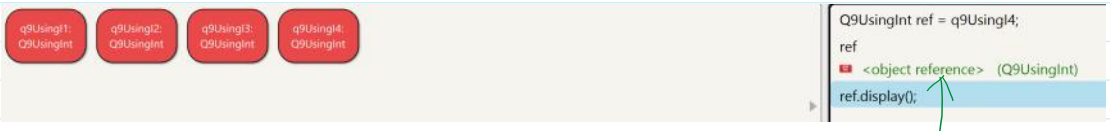




Types of Java Constructors

1. **Default Constructor(No-arg Constructor):** The default constructor does not take any parameters and is used to initialize an object with default values. If no constructor is defined, Java automatically provides a default constructor.

2. **Parameterized Constructor:** The parameterized constructor accepts parameters and is used to initialize the object with specific values at the time of object creation.
    i. **Copy constructor:** A copy constructor is a special type of constructor that is commonly utilized to create a new object by copying the values of an existing object of the same class.



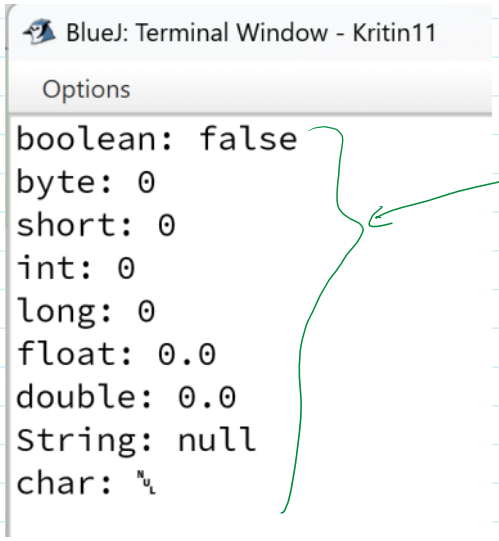
ref and q9UsingI4 is referencing to same memory allocation.
If we call the method by using "ref", the state of the object may change.

Purpose of a Default Constructor

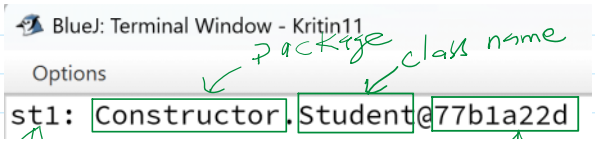
The default constructor is used to provide the default values to the object like zero to int, null to String etc., depending on the type.

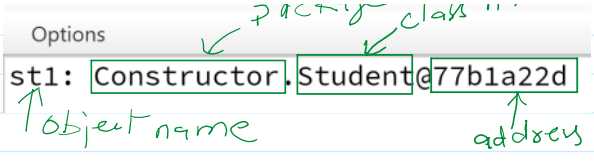
```
package Constructor;

/**
 * Write a description of class q here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
    private boolean b;
    private byte bt;
    private short s;
    private int i;
    private long l;
    private float f;
    private double d;
    private String str;
    private char c;

    public void display(){
        System.out.print("\fboolean: " + b +
                         "\nbyte: " + bt +
                         "\nshort: " + s +
                         "\nint: " + i +
                         "\nlong: " + l +
                         "\nfloat: " + f +
                         "\ndouble: " + d +
                         "\nString: " + str+
                         "\nchar: " + c);
    }

}
```

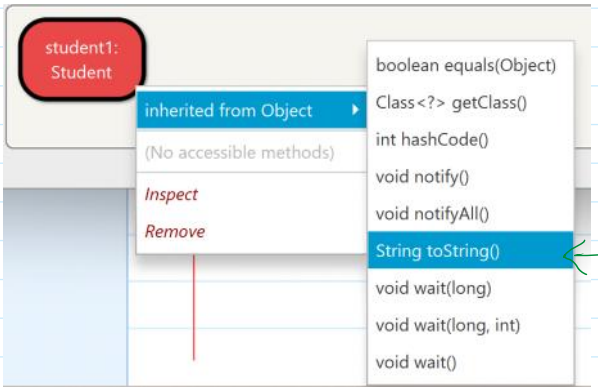
Output:



values assigned by the default constructor written by Java compiler,







"Object class is the Parent of every class"
It contains a toString() method. Object class knows the reference of child class. Hence the reference is printed on the terminal..

This function is called inside the System.out.print() method automatically.

```
package Constructor;

/**
 * Write a description of class q here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class Student
{
    //Properties are instance variable.
    private int rollNum;
    private String name;

    //Methods are called behaviors.
    //Parameterized constructor.
    public Student(int rollNum, String name){
        this.rollNum = rollNum;
        this.name = name;
    }

    //Copy Constructor
    public Student(Student st){
        this.rollNum = st.rollNum;
        this.name = st.name;
    }

    @Override
    public String toString(){
        return "[rollNum = " + rollNum + ", name = " + name + "]";
    }

    public static void main(String[] args){
        Student st1 = new Student(1, "Raman");
        System.out.print("\fst1: " + st1);
    }
}
```

Output:
st1: [rollNum = 1, name = Raman]

Java Copy Constructor

Homework

1. Custom Constructors & Object Tracking
Create a class Book that stores title, author, price, and a unique book ID auto-generated for each new object.
Requirements:
• Implement multiple constructors: default, one with title+author, and one with all fields.
• Maintain a static counter that increments for each book created.
• Add a method applyDiscount(double percent).
• Write a demo program that creates several books using different constructors and prints all information.

2. Constructor Overloading with Validation
Build a Student class with fields: name, age, and grade.
Requirements:
• Provide three constructors: default, name-only, and full constructor.
• Validate that age is between 5–120, otherwise default to 18.
• Validate grade range 0.0–4.0, otherwise default to 0.0.
• Include a printDetails() method.
Create a tester that demonstrates all cases (valid and invalid).

3. Chaining Constructors (this())
Create a class Rectangle with width and height.
Requirements:
• Use constructor chaining so the default constructor calls the one-argument constructor, which then calls the two-argument constructor.
• Add methods for area, perimeter, and resizing the rectangle.
• Show how each constructor is triggered with a test program.

4. Bank Account Simulation with Multiple Constructors
Create a BankAccount class with: name, account number, and balance.
Requirements:
• Constructor that auto-generates the account number.
• Another constructor allows custom account number (for migrating customers).
• Methods: deposit, withdraw (with insufficient balance warning), transfer.

- A driver class that simulates multiple operations between accounts.

5. Inventory System Using Array of Objects
   Write a Product class (id, name, quantity, price) with overloaded constructors.
   Write a second class Inventory that uses an array of Product objects.
   Requirements:
   - Add a method to add new products, update quantities, compute total inventory value, and search by name.
   - Demonstrate your system with at least five products.

6. Library Management with Constructors and Encapsulation
   Create classes Author and Book.
   Requirements:
   - Author has name, email, and gender.
   - Book has title, price, and an Author object.
   - Implement constructors for both classes.
   - Add methods to display complete book info including author details.

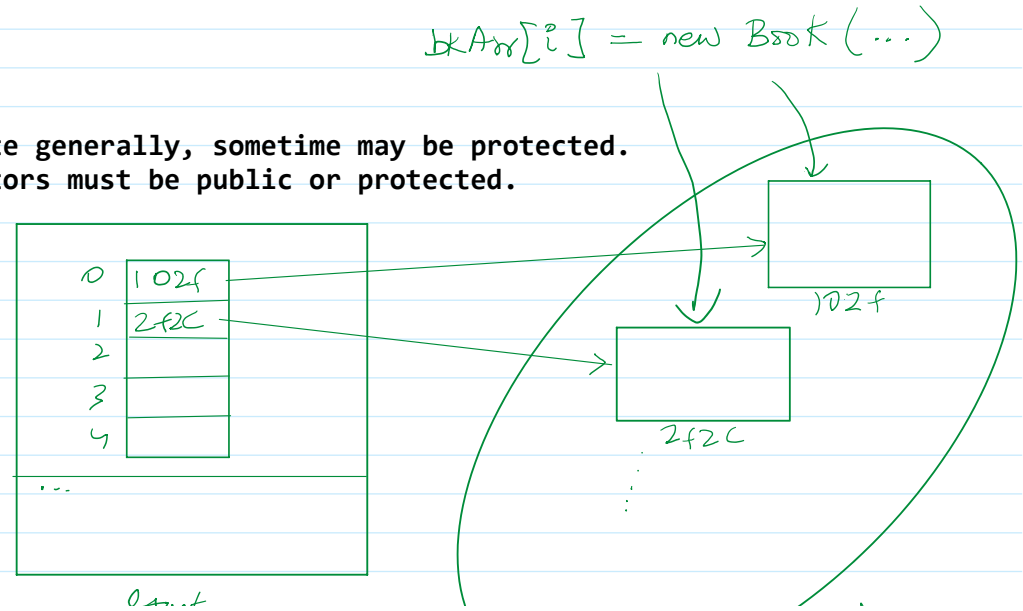
   Create a main program that builds several books using different authors.

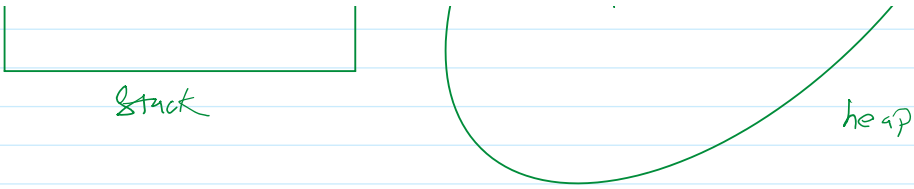
7. Object Copying (Copy Constructor Scenario)
   Create a class Point (x, y).
   Requirements:
   - Implement a constructor that initializes x and y.
   - Implement a copy constructor that copies coordinates from another Point object.
   - Add a method to move the point by dx, dy.
   - In main, create a point, create its copy, modify the original, and show how the copy behaves.

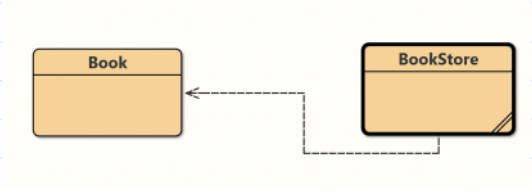
8. Employee Payroll with Constructor Logic
   Create an Employee class with name, hourlyRate, and hoursWorked.
   Requirements:
   - Overload constructors: default, name-only, name+rate.
   - Pay should be computed using a method that includes overtime (hours > 40 paid at 1.5x).
   - Add a method to print a formatted pay slip.
   - Demonstrate using different employees.

Java community rule:
1. Instance variable are private generally, sometime may be protected.
2. Member methods and constructors must be public or protected.

```
Book bkArr[] = new Book[5];
```








```
package Constructor.HomeWork;

/**
 *  1. Custom Constructors & Object TrackingCreate a class Book that stores title,
author, price, and a unique book ID
 *  auto-generated for each new object.
 *  Requirements:
 *  • Implement multiple constructors: default, one with title+author, and one with all
fields.
 *  • Maintain a static counter that increments for each book created.
 *  • Add a method applyDiscount(double percent).
 *  • Write a demo program that creates several books using different constructors and
prints all information.

 * @author (your name)
 * @version (a version number or a date)
 */
public class Book
{
    private String title, author;
    private double price;
    private int bookId;
    private static int count = 0;
    public Book(String title, String author, double price){
        this.title = title;
        this.author = author;
        this.price = price;
        this.bookId = count + (count+1) * new java.util.Random().nextInt(1, 100);
        count++;
    }

    public Book(String title, String author){
        this(title,author, 0);
    }

    public Book(){
        this("", "");
    }
```

```
    public double applyDiscount(double percent){
        return price * percent;
    }

    public void input(String t, String a, double p){
        title = t;
        author = a;
        price = p;
    }

    public void setPrice(double p){
        price = p;
    }

    public void display(){
        System.out.print("\n------Book Information-----\n"+
            "Title: " + title + "\n"+
            "Author: " + author + "\n" +
            "Price: \u20b9"+price+"\n"+
            "\nBook ID: " + bookId+"\n");
    }

    public static int numberOfBooks(){
        return count;
    }
}

package Constructor.HomeWork;
import java.util.*;


/**
 * Write a demo program that creates several books using different constructors and
prints all information.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class BookStore
{
    public static void main(String[] args){
        Book bk1 = new Book("Java", "Pandey", 200);
        Book bk2 = new Book("Maths", "M L Agarwal");
        Book bk3 = new Book(); //calling default constructor

        System.out.print("\fNumber of Book objects: " + Book.numberOfBooks());
        bk3.input("Physic", "H C Verma", 600);
        bk2.setPrice(560);
        bk1.display();
        bk2.display();
        bk3.display();

        Book bkArr[] = new Book[3];
        int i;
        Scanner sc = new Scanner(System.in);
```

```
        for(i = 0; i < bkArr.length; i++){
            //allocating memory for each object stored in the array
            System.out.print("\nEnter title: ");
            String t = sc.nextLine();
            System.out.print("Enter author name: ");
            String a = sc.nextLine();
            System.out.print("Price: \u20b9");
            double p = sc.nextDouble();
            sc.nextLine(); //to remove keyboard buffer
            bkArr[i] = new Book(t,a,p);
        }

        for(i = 0; i < bkArr.length; i++){
            System.out.print("\n" + bkArr[i]); //it will print the references of each
object;
            bkArr[i].display(); //it will display the information of each object
        }

    }
}

Output:
Number of Book objects: 3
------Book Information-----
Title: Java
Author: Pandey
Price: ₹200.0

Book ID: 84

------Book Information-----
Title: Maths
Author: M L Agarwal
Price: ₹560.0

Book ID: 65

------Book Information-----
Title: Physic
Author: H C Verma
Price: ₹600.0

Book ID: 236

Enter title: HTML
Enter author name: resnic
Price: ₹350

Enter title: JavaScript
Enter author name: Hall
Price: ₹450

Enter title: CSS
Enter author name: Haliday
Price: ₹430
```

Input by the user

```
Enter title: CSS
Enter author name: Haliday
Price: ₹430

Constructor.HomeWork.Book@2490a45b
------Book Information-----
Title: HTML
Author: resnic
Price: ₹350.0

Book ID: 39

Constructor.HomeWork.Book@67003d5d
------Book Information-----
Title: JavaScript
Author: Hall
Price: ₹450.0

Book ID: 499

Constructor.HomeWork.Book@5ae05892
------Book Information-----
Title: CSS
Author: Haliday
Price: ₹430.0

Book ID: 569
```